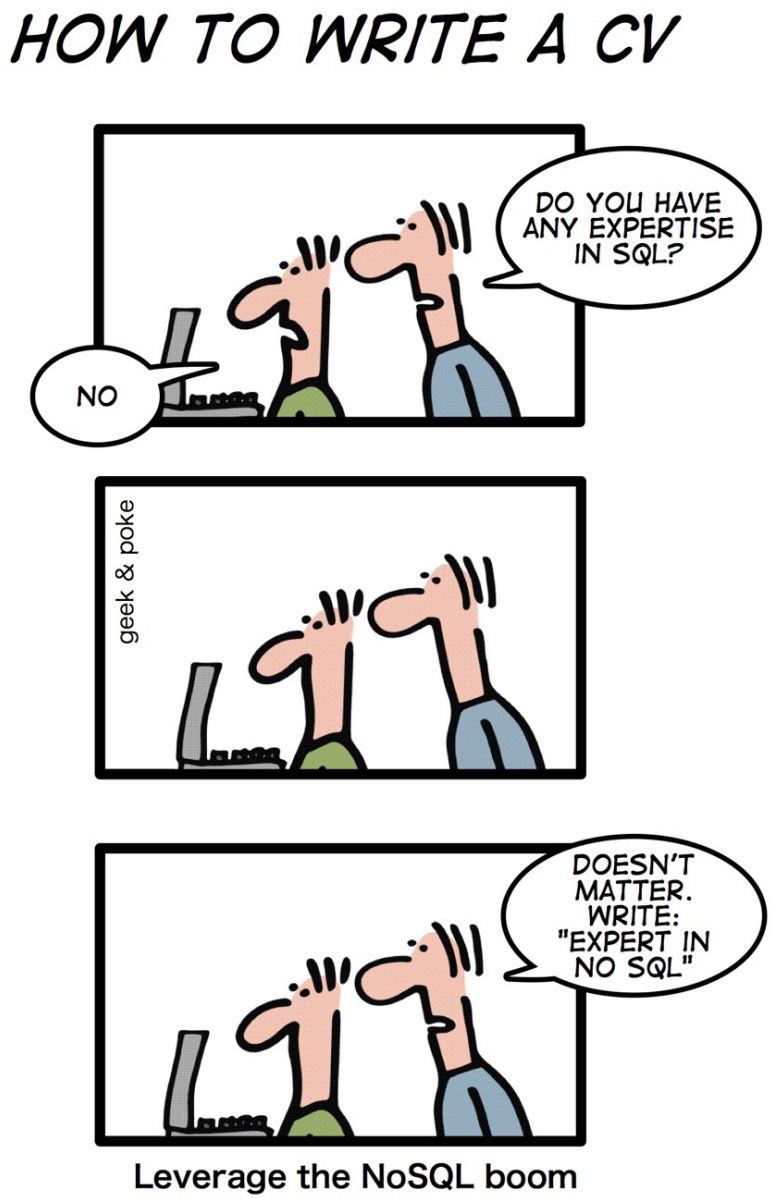
INTRODUCTION TO
NOSQL DATABASES



Source: geekandpoke.typepad.com/geekandpoke/2011/01/nosql.html

Finished overview of traditional RDBMS and PDBMS

Traditional DBMS

Structured data (relational), centralized, disk-based, OLTP workloads, ACID

PDBMS

Structured data (relational), ~~centralized~~ parallel, disk-based, ~~OLTP~~ MIXED/OLAP workloads, ACID

NoSQL

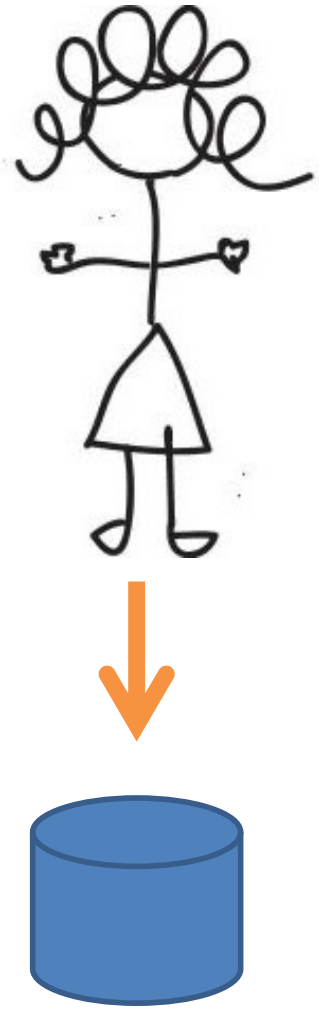
not only relational disk or RAM-based OLTP

Structured data (~~relational~~), parallel, ~~disk-based~~, ~~MIXED/OLAP~~, ~~ACID~~

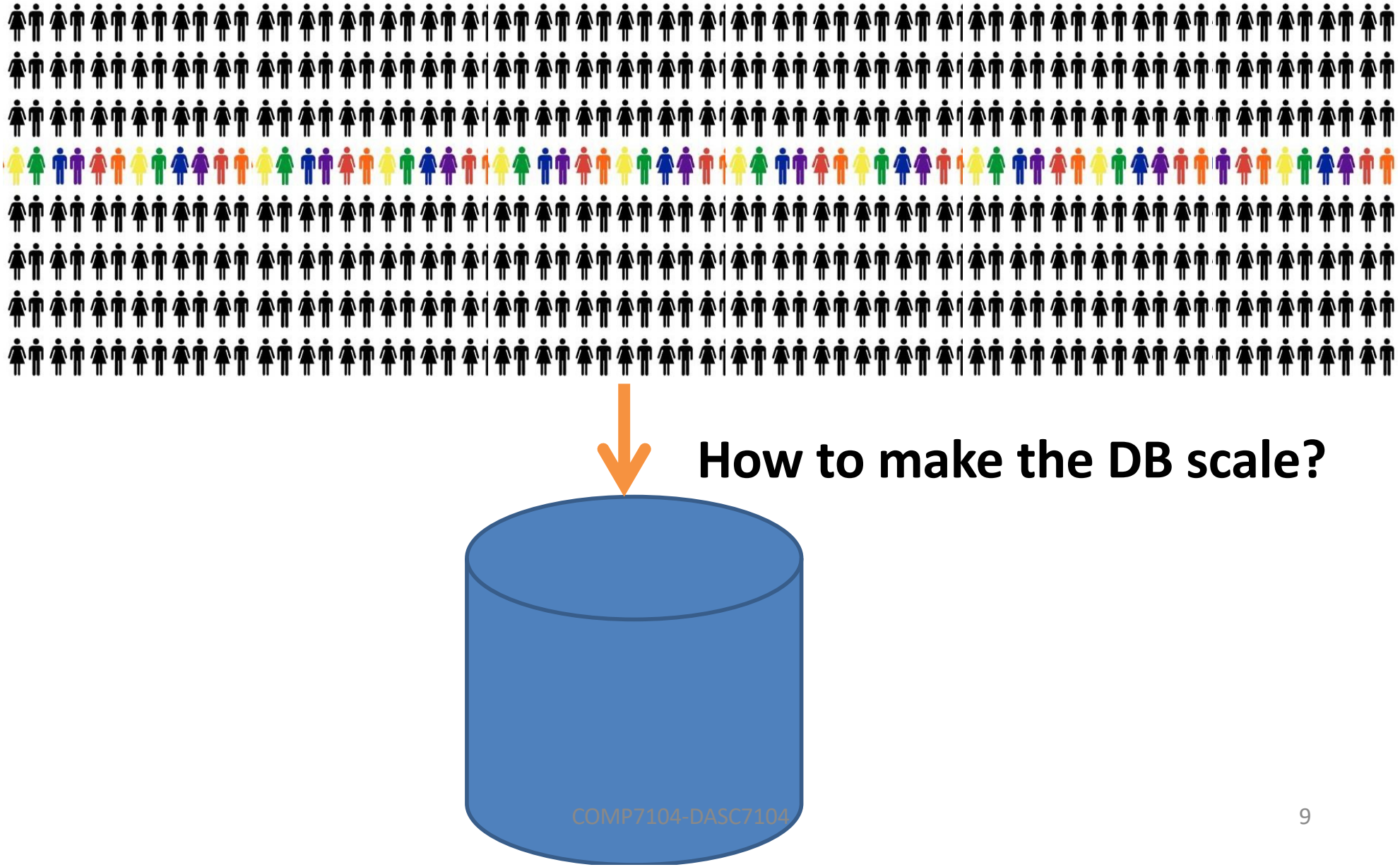
How the Web changed the database world

- Databases used to be the domain of corporations with limited amounts of data and users
  - Very valuable information, but not a lot of it
  - Important users, but not many of them
- In the modern web-driven world (Web 2.0)
  - Enormous amounts of data are being generated
  - Millions of users are interested in that data
  - Rise of cloud-based solutions such as Amazon S3
  - Open-source community (good for startup costs)
- NoSQL intention: modern web-scale databases (movement began early 2009)

Going from here…

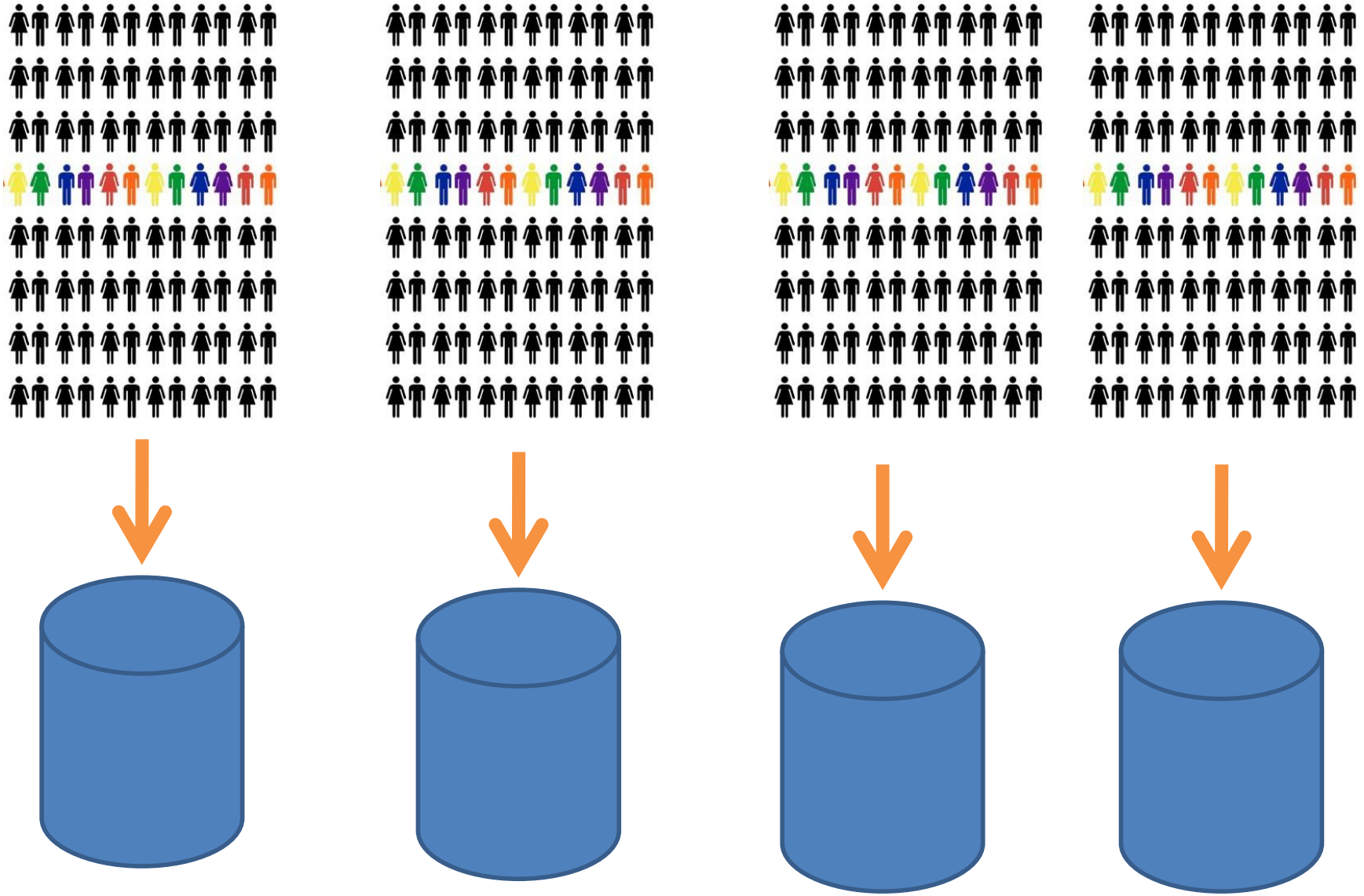


… to here



How to make the DB scale?

Partition, distribute and *replicate* the data



Motivation for NoSQL stores

- We want to keep track of *mutable* state in a *scalable* manner
- Assumptions:
  - State organized in terms of many “records”
  - State unlikely to fit in a machine, must be distributed
  - **High rates of “simple” read / write**

NoSQL designed for

Scaling simple OLTP-style application loads to large volumes of data and thousands / millions of users → good *horizontal scalability* for *simple read/write DB operations* distributed over many servers

- to do updates as well as reads
- in contrast to traditional DBMSs (centralized, cannot scale) and data warehouses / PDBMSs (designed mostly for analytical workloads)

What are « simple operations »

- Reading or writing one or a small number of related records in each operation
  - e.g. key lookups
  - contrast to complex queries, joins, or read-mostly access (Big Data analytical batch processing)
- Inspired by Web 2.0, where millions of users may both read and write data in simple operations
  - e.g. search and update multi-server databases of electronic mail, personal profiles, web postings, wikis, customer records, online dating records, etc

Horizontal scaling

Shared-nothing horizontal scaling → ability to distribute both the data and the load of these simple operations over **many nodes**

- no RAM or disk shared among the nodes
- not “vertical” scaling, where system uses more cores and/or CPUs that share RAM and disks

Core data organization ideas in NoSQL

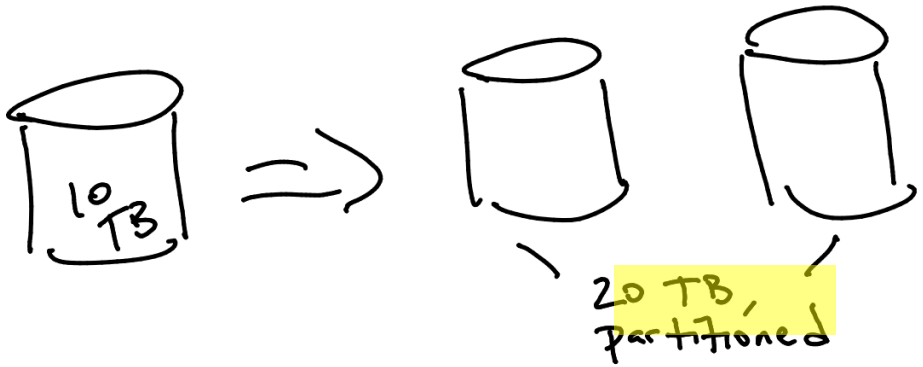
- Partitioning (horizontal partitioning aka sharding)
  - For scalability (data)
  - For latency (fast response)
- **Replication** (transparent!)
  - For robustness (availability)
  - For throughput (workload scalability)

  (better when used for non-volatile / read-only data)
- In-memory caching
  - For latency (fast response)

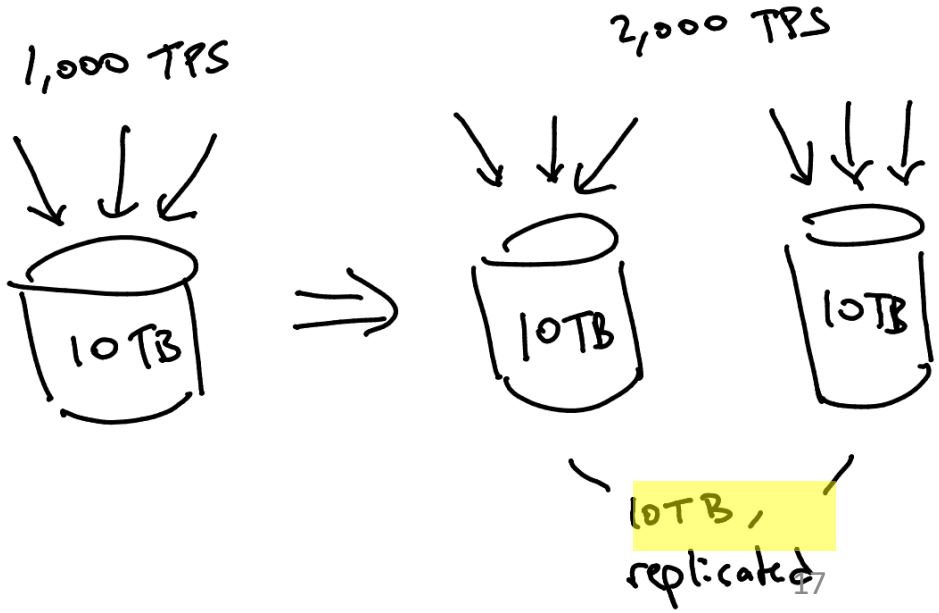
REPLICATION

Why replicate data (1)? Scale

- Does NOT help with data scaling
  - That's what partitioning ("sharding") is for !



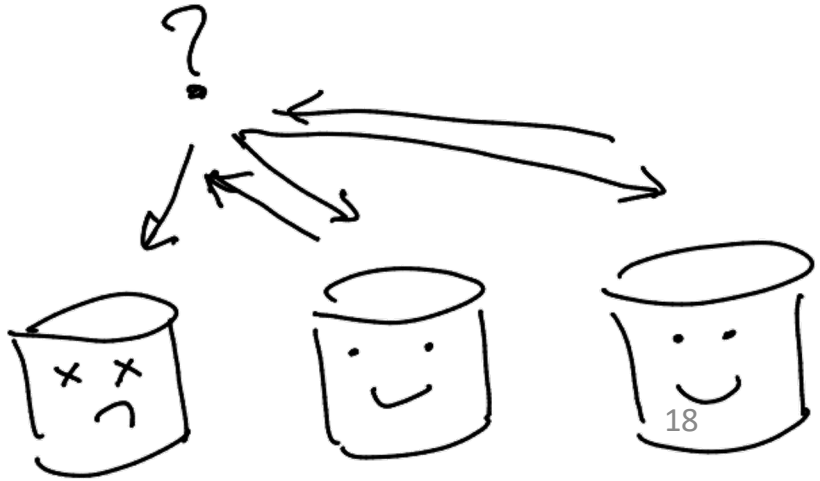
- DOES help with (read) workload scaling!
  - Even on a small database
  - Load balancing



Why replicate data (2)? Availability

Replication increases availability !

- If one replica cannot answer, another can
- Tolerate one node's temporary unavailability
  - Software crash, transient workload spike, JVM GC
- Survive catastrophic failures
  - Avoid correlation: place replicas on a different rack, different datacenter



Why replicate data (3)? Locality

- Replication reduces latency
  - Choose a “nearby” server
    - Particularly for geo-distributed DBs
  - Ask many servers and take the 1st response

Other replication details

- Replication can be at various granularities:
  - database, table, partition, tuple
- Degree and choice of replication flexible
  - 3 replicas is a good start; why ?
  - choose locations in different “fault domains” for availability
    - different racks, datacenters, continents
  - replication factor could vary across objects
    - “hot objects” replicated to more nodes
    - improves read workload balancing
  - some scheme needed to “route” queries to replicas
    - with load balance and fault tolerance
    - Consistend Hashing / Distributed Hash Tables (DHT)

More replicas → more problems…

- How do we keep replicas in sync (problem of *replica consistency*)?
- How do we synchronize transactions across multiple partitions?

Recall how do PDBMS handle transactions on distributed data ?

- Transactions on a single machine: relatively easy !
- Maybe partition tables to keep transactions on a single machine → also relatively easy
  - Example: partition Facebook posts by Facebook user
- Transactions that require data from multiple machines?
  - Example: transactions involving several Facebook users

Solution: Two-Phase Commit (2PC)

  - But 2PC is blocking and slow, what if the coordinator dies…
    - Solutions: other protocols (3PC, Paxos, Raft), even slower!

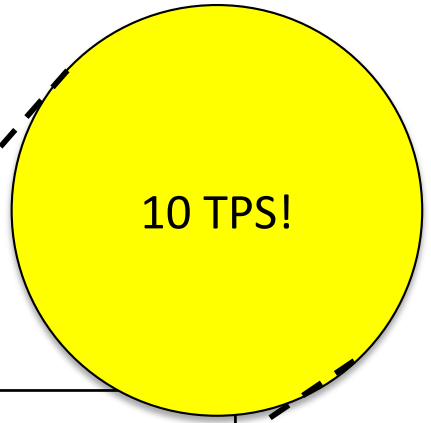
Is 2PC really too slow?

- How expensive is it, really, to hold a vote?
  - Raw latencies depend on your network
    - geo-replication and speed of light
    - vs. modern datacenter switches
  - But best-case behavior is not a good metric !

- Depends on your machines' delay distribution
  - Latency to get responses from **all** participants?
    - The ***max*** latency is the right metric, not the average
    - "Straggler" sensitive (so-called tail effects)
  - Hardware: switches, magnetic disk vs. flash, etc.
  - Software: GC in the JVM, etc.

Example: 2PC in Google Spanner

- Google Spanner / F1 DBMS uses 2PC (and other stuff)
- Latencies (slowed by speed of light)
  - 7 times round the world per second



operation	latency (ms)		count
	mean	std dev	
all reads	8.7	376.4	21.5B
single-site commit	72.3	112.8	31.2M
multi-site commit	103.0	52.2	32.1M

Quick quizz

- 2PC is unattractive at cloud scale because:
  A. Coordinator failure can cause unavailability
  B. The average latency for 2PC in a data center is a bottleneck
  C. The max latency for 2PC in a data center is a bottleneck
  D. The average latency across the globe for 2PC is a bottleneck

Replication – update (write) possibilities

- Update sent to all replicas at the same time
  - To guarantee consistency you need something like 2PC (slow)
- Update sent to a master
  - Replication is synchronous (also slow)
  - Replication is asynchronous
  - Combination of both
- Update sent to an arbitrary replica

All these possibilities involve tradeoffs in a range of “eventual replica consistency”.

Master-slave & syncronous writes

- All writing is done by request to Master, who will dispatch to slaves
- Ack sent to client only when copies in sync
- Strong consistency, as any read on that data, regardless from which node, gives the same version
- But slow …

Master-slave & asyncronous writes

- Same master-slave topology, but asyncronous writes (Ack sent to client w/o waiting for copies to apply the changes)
- Weak consistency (eventual consistency): possible to write by talking to master, read from a slave
- If the read is done before replicas in sync, stale (before update) version of data may be returned
- Often in NoSQL → delay between a write and a « consistent read »

Multi-node & asyncronous writes

- Write can be done at an arbitrary node (improves scalability)
- Pb: concurrent writes of same object
- When replicas are synchronized, may discover conflicting versions !
- Conflicts must be reconciled at application level (no default mechanisms in the DB)

Quick quizz

Replication or partitioning (or both)?

A. Enables workload scaling
B. Enables data scaling
C. Depends on 2PC for correctness
D. Helps with availability

NOSQL OVERVIEW

Recall what RDBMS / PDBMS provide

- Relational model with schemas
- Powerful, flexible, declarative query language
- Transactional semantics (even distributed): ACID
- Rich ecosystem, lots of tool support

RDBMS / PDBMS downsides

- Must design up front, painful to evolve, high cost
- 2PC is slow and has limitatios
- Consistency limits the scalability of RDBMSs
- Not good for large volume (PB) of data with variety of data types (images, videos, text)

Remember what is scalability: ability to handle *changing demands* by adding/removing resources

NoSQL: DBMS features *à la carte*?

- What if I'm willing to give up replica consistency for scalability ?
- What if I'm willing to give up the relational model / SQL for something more flexible ?
- What if I just want a cheaper solution ?

Enter… NoSQL (Not Only SQL)

- Note: would have been better called "Not Only Relational"

NoSQL systems

New systems choosing to use or not use several concepts learnt so far, e.g.,

- NoSQL system that doesn't support transactions, or may not use locks but multi-version CC
- Key—value API instead of declarative languages (SQL)
- ….

Hence it is important to understand the basics on RDBMS even if they are not always / all used in some new systems ! And learn to compare !

NoSQL definition

NOSQL: **NO SQL** or **Not Only SQL**

Say No! to schemas and declarative queries

A simple definition:

NOSQL is a DB technology that does not follow the well-established relational database principles.

A pragmatic response to growing scale of databases and the falling prices of commodity hardware.

NoSQL key features (and bugs?)

1. Horizontally scale “simple operations”
2. Replicate & distribute data over many servers
3. Simple call API (less expressive than SQL, no joins)
4. Weaker concurrency model than ACID (weak isolation)
5. Efficient use of distributed indexes and RAM
6. Flexible schemas / schema-less data models / schema-on-read, non-relational
7. Open source, cheap, faster development

Examples of choices in NoSQL systems: Transaction support

a) Support transactions (all-or-nothing sequence of DB operations)
b) Do not support
c) In between
   - support local transactions only within a single object or partition / shard
   - shard = a horizontal partition of data in a database

Examples of choices in NoSQL systems: Concurrency control

- Locks
  - some systems have one-user-at-a-time read or update locks
  - others provides locking at a field level
- Multi-version concurrency control (MVCC)
- Other
  - pre-analyze transactions to avoid conflicts (no deadlocks and no waits on locks)
- None
  - do not provide atomicity of updates of replicas
  - multiple users can edit in parallel
  - no guarantee which version you will read

Examples of choices in NoSQL systems: Storage medium

- Storage in RAM
  - snapshots or replication to disk
  - poor performance when overflows RAM
- Disk storage
  - caching in RAM

Examples of choices in NoSQL systems: Replication

Whether mirror copies (replicas) are always in sync

a) Synchronous

b) Asynchronous

- faster, but updates may be lost in a crash

c) Combination of both

- local copies synchronously, remote copies asynchronously

Examples of choices in NoSQL systems: Data model

- Unstructured data
  - Key-value stores


- Semi—structured data
  - Document stores

- Structured data
  - Extended relational

- Graph data (Neo4J)

NoSQL in summary: mainly two rejections

1. **Saying no to schemas and declarative queries**
   - Not agile to define schemas in advance
   - SQL is a “hoop to jump through” from my “real” language
   - Instead, for instance, key-value stores
     - put(key, val), val = get(key)
   - Sometimes the values are (JSON) document (MongoDB)
2. **Saying no to serializable transactions *(relaxing transactions)***
   - Much easier to scale massively without them!
     - Replicate aggressively
     - **No 2PC: don’t worry about partial failure**
   - Many use cases don’t need multi-object transactions anyway

Quick quizz

- **T/F**: the two main “NOs” of NOSQL are independent. E.g. you can have:
  - serializability + SQL
  - no serializability + SQL
  - serializability + put/get API
  - no serializability + put/get API

RELAXING REPLICA CONSISTENCY

Without 2PC → data ambiguities

- Per-object ambiguities
  - You may not read the latest version
  - Writers can conflict
    - Write the same key in different places
    - ‘Conflict Resolution’ or ‘Merge’ rules apply:
      - E.g., last / first writer wins, but what clock do you use?
      - E.g., keep all values in a set (and let application decide)
- Across objects ambiguities (no serializability)
  - Typically, no transactional guarantees

Consistency as a continuum with tradeoffs

Strong replica consistency:

- Ideal: all copies of item X updated together/at once, atomically
- Really: readers ***see*** values of X that they would see w/o concurrency
  - **Aka linearizability** (a.k.a. "atomic consistency")

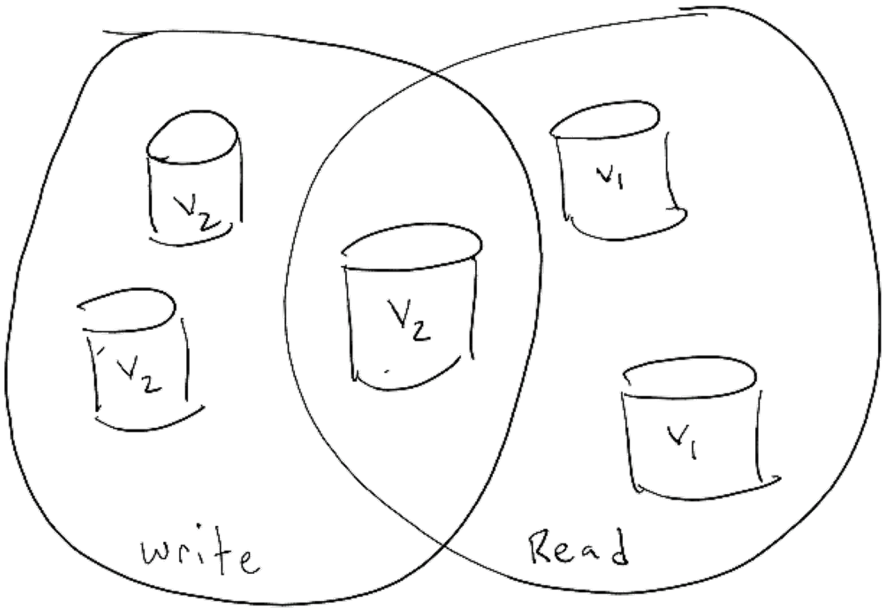
Eventual replica consistency: can mean anything…

Quorum replica consistency: practical "strong" consistency

Quorum consistency

- Assume writes are globally timestamped
  - <local-clock, node-ID>
  - synchronous node-level reads and writes
- Want to ensure that all readers see the same ("consistent") value
  - Even in the face of delayed replication
  - Again: this is *NOT* the C in ACID! It's linearizability.
- Idea 1: Write to all
  - Can read any 1 copy and get the "latest"
- Idea 2: Read from all
  - Can label as "latest" any copy
- Idea 3: Read & write a majority
  - Read guaranteed to see "latest" on at least one node

Quorum consistency



- More generally:
  - Write to a write-quorum of *w* nodes
  - Read from a read-quorum of *r* nodes
  - Ensure *r + w > N* (#nodes)
    - Optional: ensure *w > N/2* and you don't need write timestamps: no write/write race conditions
- Assumption (big assumption!)
  - The set of nodes in the system (membership) is static
  - How do we handle failures? New nodes?

NoSQL → weak / eventual consistency

- E.g. Amazon, Facebook, LinkedIn, Twitter
  - Shopping, posting, connecting
- **Latency** and **availability** both crucial
  - Replication is critical
  - Favors multi-master with **loose** quorum
  - Replica consistency becomes frustrating
- Becomes quite natural to give up strong replica consistency !
    - All the more so serializability !

Eventual consistency (EC)

"if no new updates are made to the object, eventually all accesses will return the last updated value"

-- Werner Vogels, "Eventually Consistent", CACM 2009

Which in practice means...??

Problems with eventual consistency

- Reads can get non-deterministic versions based on what replicas they contact
- Replica values may diverge permanently
- You may not be able to read your own writes
- Writes can be applied out-of-order w.r.t. reality

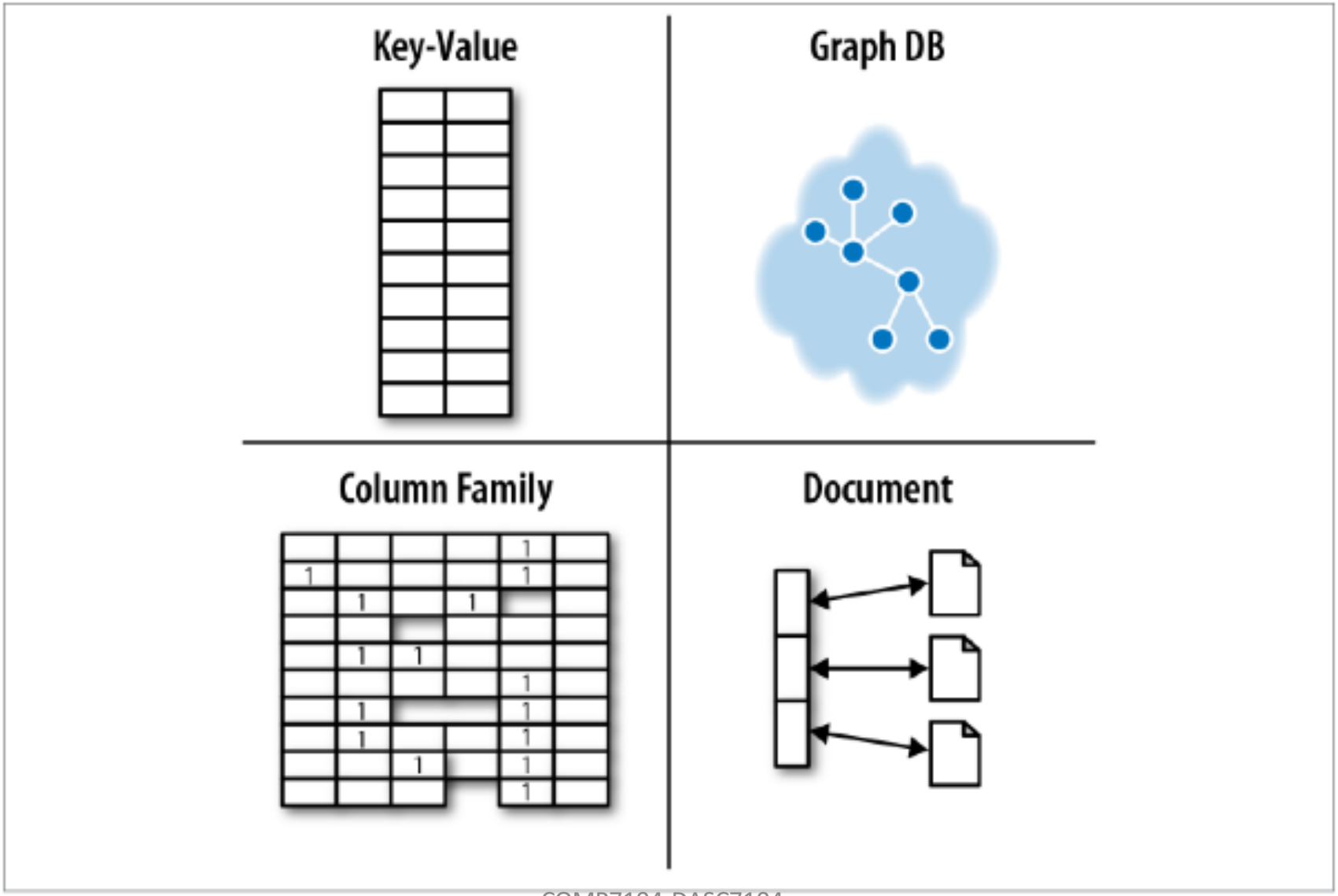
EC puts burden on app developers

Application-level reasoning: the application needs to manage race conditions in its own (often parallel) setting

- Given the tools you have
  - Java/Eclipse, C++/gdb, etc.
- Convince yourself your app is correct
  - No race conditions? Fault tolerant?
- Convince yourself your app will *remain* correct
  - Even after you are replaced
- Convince yourself the app plays well with others
  - Does your eventual consistency affect someone else's serializable data?

NOSQL EXAMPLES

Major types of NoSQL databases



KEY-VALUE STORES - DYNAMODB

Key-value stores

- Built on a technology you know and use all the time
  - Java Hashmap for example
- Put(Key,Value)
- Value = Get(Key)
- Pros / Cons ? A file system or a database ?
- Examples
  - DynamoDB
  - Redis – in memory store
  - Memcached
  - and 100s more…

DynamoDB (2007)

- **Amazon's highly-available** key-value store → pioneered eventual consistency for higher availability & scalability
- Query model: no need for relational schema, simple read/write based on primary key (enough – at the time at least – for most Amazon Services)
- Eventual consistency (in exchange for high availability), no isolation, only single key updates
- Replication
- Able to meet stringent SLAs on latency and throughput
  - « Respond within 300ms for 99.9% of its requests »
- Always writable & resolve update conflicts during reads
- Decentralized, no single point of failure (SPF)

Key-value stores: operations

- Very simple API:
  - Get – fetch value associated with key
  - Put – set value associated with key
- Optional operations:
  - Multi-get
  - Multi-put
  - Range queries
- Consistency model:
  - Atomic puts (usually)
  - Cross-key operations: sometimes…

Key-value stores: implementation

- Option 1: non-persistent
  - Just a big in-memory hash table
- Option 2: persistent
  - Wrapper around a traditional RDBMS

What if data doesn't fit on a single machine?

Simple solution: partition!

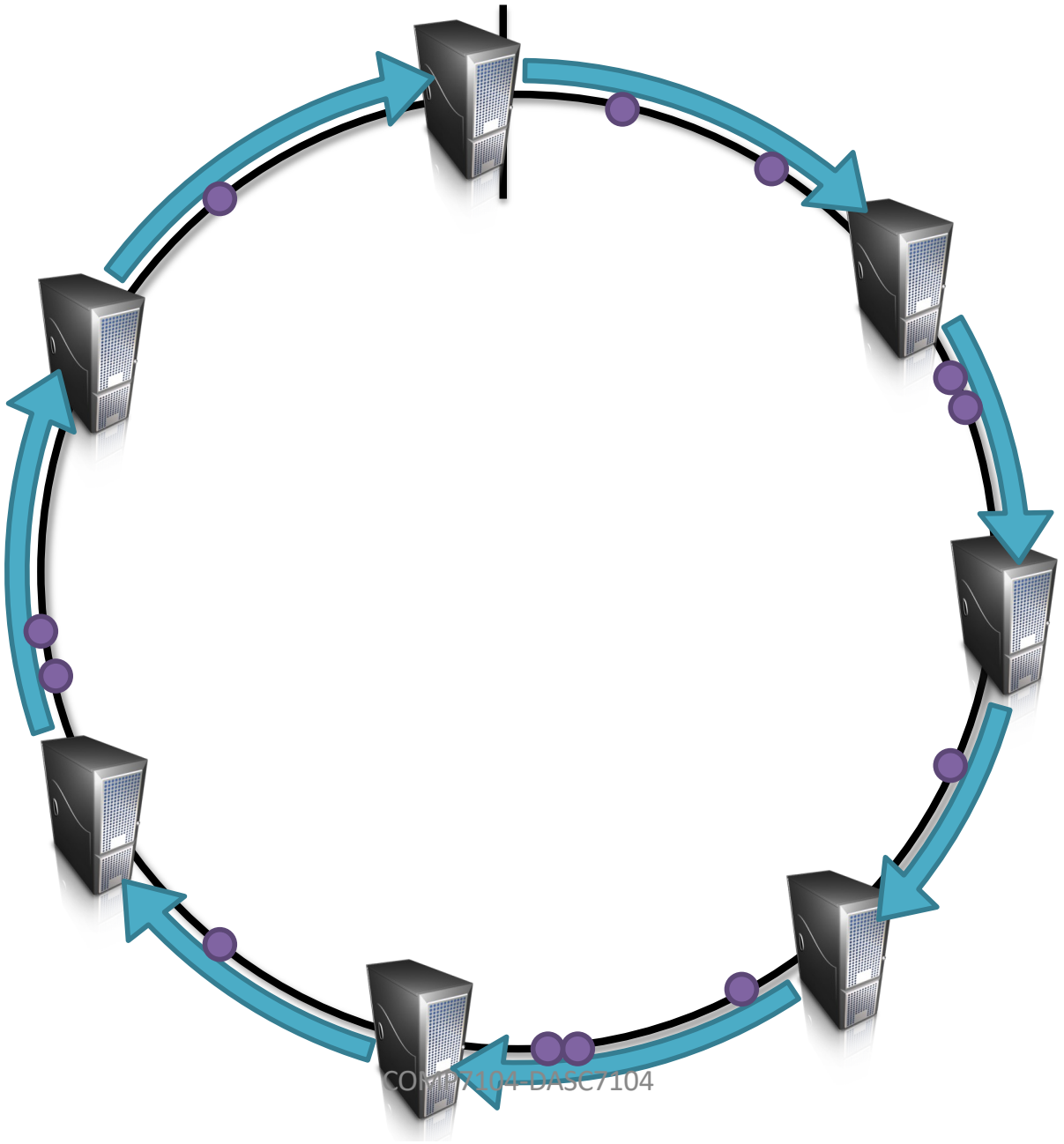
- Partition the key space across multiple machines
  - Let's say, **hash partitioning**
  - For *n* machines, store key *k* at machine *h(k)* mod *n*
- Issues:
  1. How do we know which physical machine to contact?
  2. How do we add a new machine to the cluster?
  3. What happens if a machine fails?
  4. What if we also want to replicate ?

Solution

- Hash the keys
- Hash the machines also !

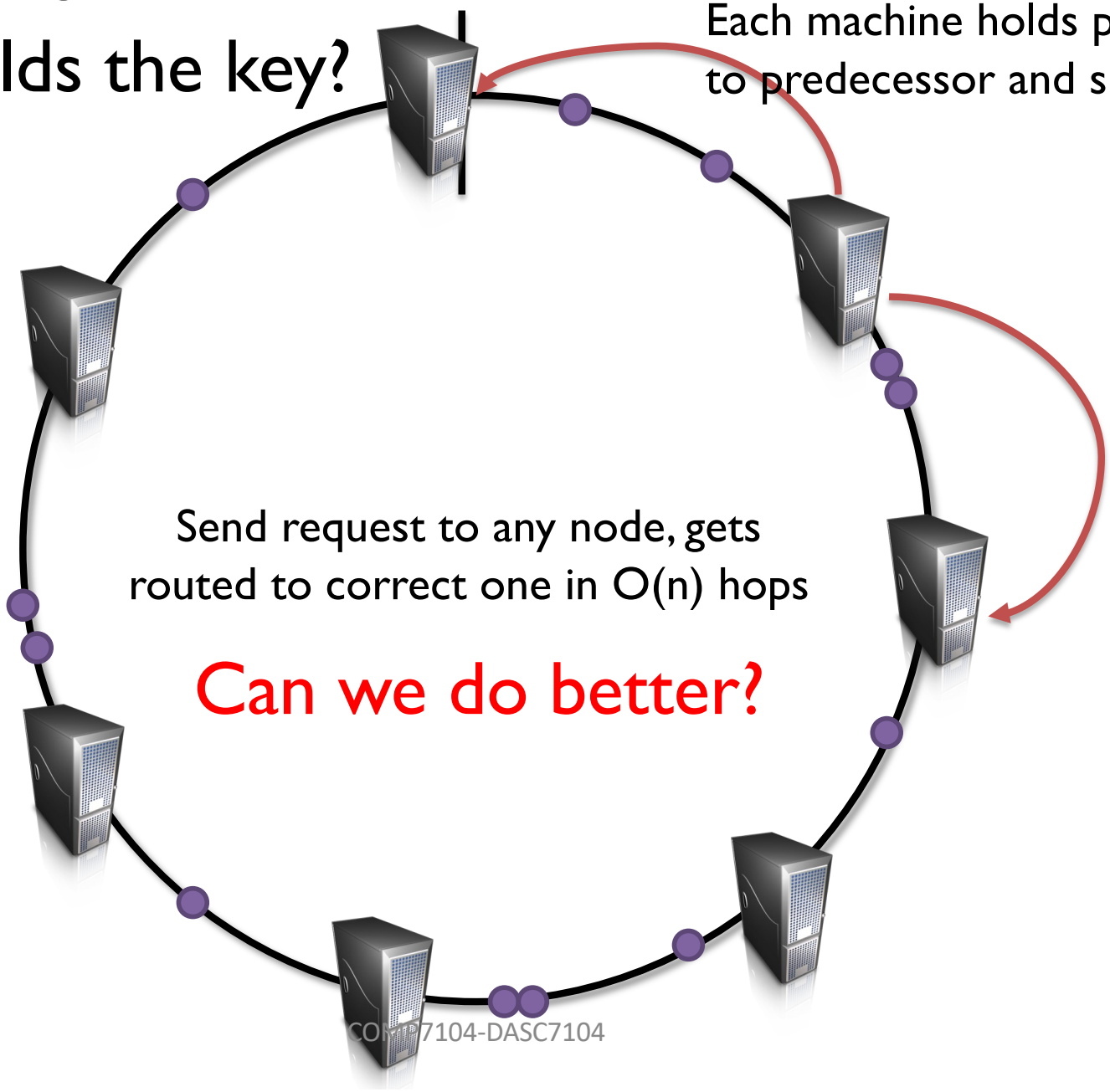
Distributed Hash Tables (DHTs) !

DHTs

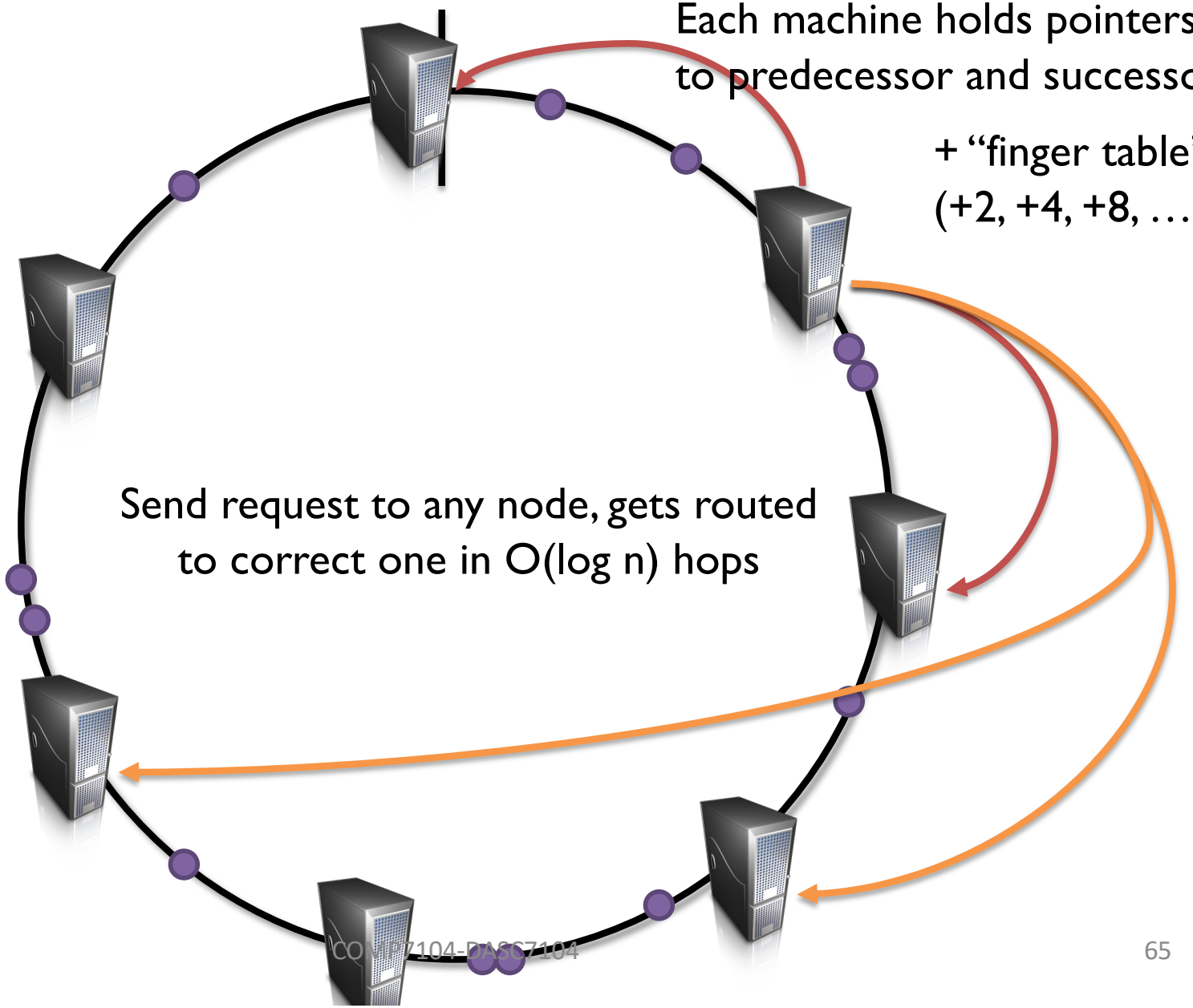


DHTs

Routing: which machine holds the key?



Routing: which machine holds the key? DHTs



DHTs

Routing: which machine holds the key?

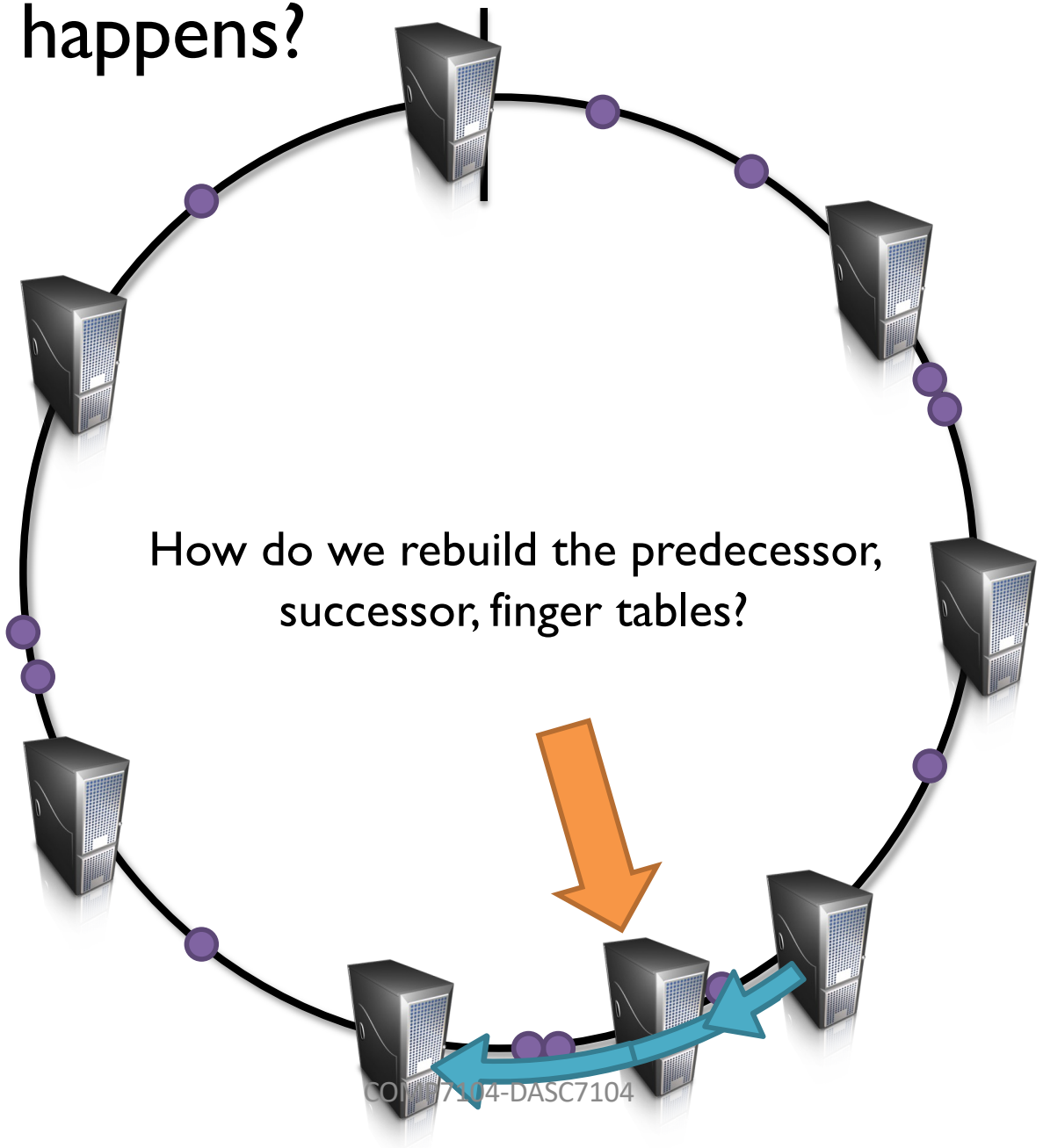
Simpler Solution

Service Registry

DHTs

Stoica et al. (2001). Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications. *SIGCOMM.*

New machine joins: what happens?

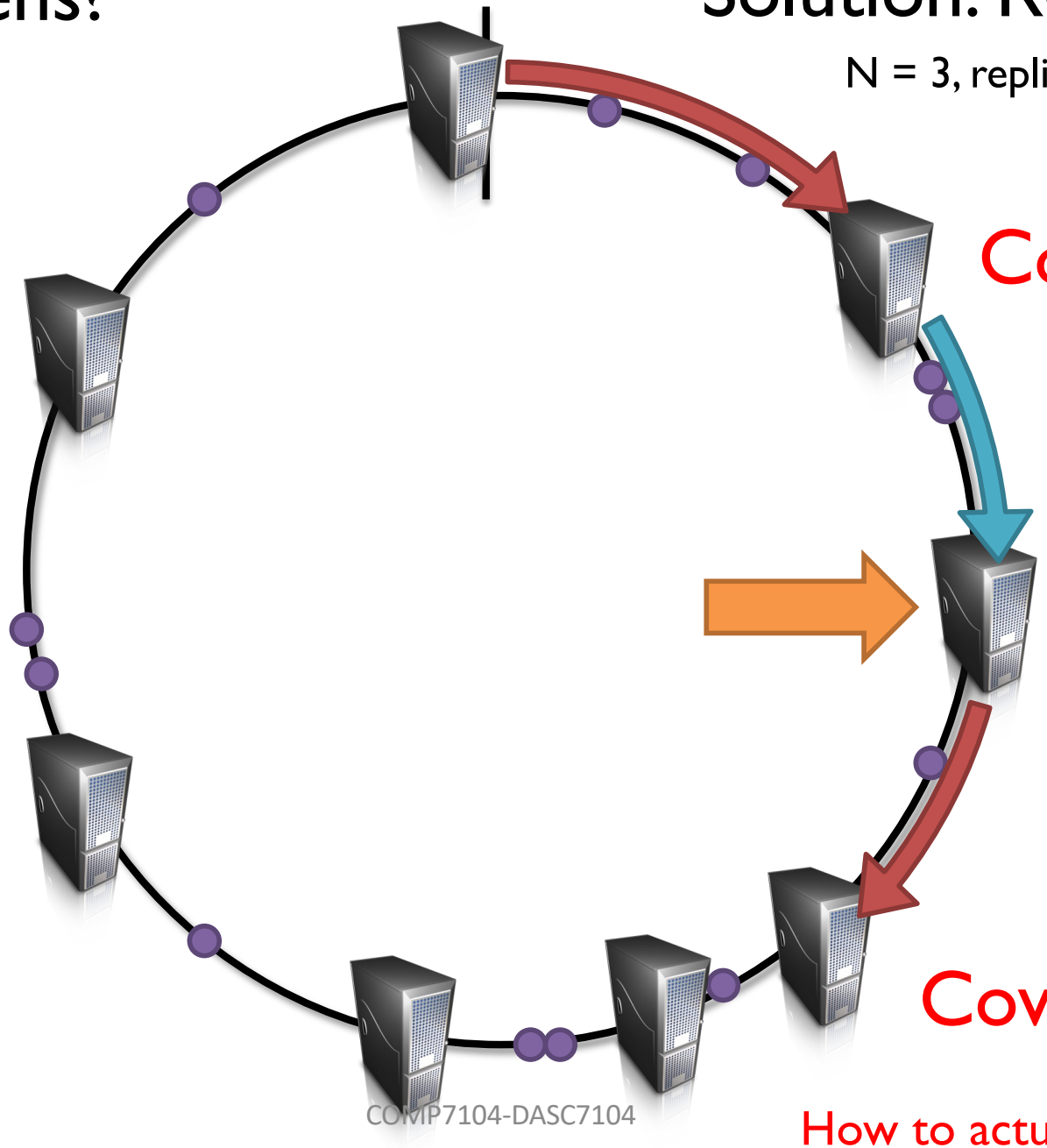


DHTs

Machine fails: what happens?

Solution: Replication

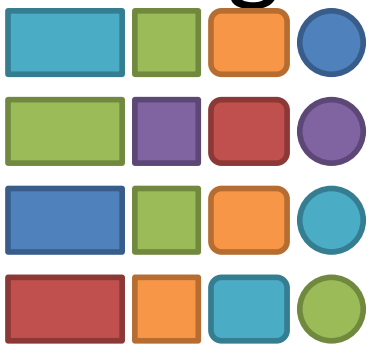
N = 3, replicate +1, –1



How to actually replicate?

WIDE-COLUMN STORES – CASSANDRA

Recall column stores
(Decomposed Storage Model - DSM)



Row store

Column store

(rowId/key, record)

vs

(rowId/**key**, attribut)

Wide-column stores

Taking further the column-oriented (DSM) idea:

- accommodate **multiple attributes / columns** per key in a distributed data structure
- columns grouped into column families (super-columns) – i.e., columns that have sub-columns
- *Key: <table, row, column-family, column, timestamp>*
  - all table accesses via this key
  - value: e.g., uninterpreted array of bytes
- Stores: Bigtable & HBase, Cassandra

Apache Cassandra: initial motivation

- Lots of data at Facebook
  - Copies of messages, reverse indices of messages, per user data, etc
- Many incoming requests resulting in a lot of random reads and random writes
- No existing production ready solutions in the market meet the requirements

Cassandra genesis

- Google Bigtable (2006)
  - consistency model: strong
  - data model: wide-column / sparse map
- Amazon DynamoDB (2007)
  - O(1) DHT (distributed hash table)
  - consistency model: client tunable

Cassandra ~= Bigtable + DynamoDB

Cassandra introduction

- Initially data model based on Google's BigTable, evolved into an extended relational model (N1NF = Non First Normal Form)
- Own language for schema definition and querying CQL (no joins)
- Inspired by Dynamo's distribution by hashing
- Fault tolerance based on replication on multiple nodes

Cassandra design goals

- High availability structured key-value store
- Incremental scalability
- Optimistic replication, i.e., enables eventual consistency (trading strong consistency for high availability)…
- … but also “knobs” to tune tradeoffs between consistency, durability, latency (fast random read/write)
- Low total cost of ownership, minimal administration
- No SPOF – structured storage over a P2P network

Cassandra – No SPOF

- Certain nodes are entry nodes in the hash ring, called seeds
- But all nodes can answer queries on behalf of client applications
- Routing table is copied at each node, so queries can be directed to where the relevant data is found
- Gossip is used for arrival / departures of nodes, maintaining the local routing table

Data model: relational + complex types

- Databases (called *keyspaces*) have tables (a.k.a. *column-families*)
- A table has rows (a.k.a. *documents*), consisting of columns (key-value pairs), with simple or complex values
- Perspective A: it's extended relational, going against the 1st rule of normalization (atomic types)
- Perspective B: each row is a structured document → close to JSON, but strongly typed !

Key-value pairs (columns)

- The basic data structure is the (Key, Value) pair (as in Dynamo)
  - Key: an identifier
  - Value: atomic (integer, string) or complex (dictionary, set, list)
- A document (row) represents a row key associated to a set of (key, value) pairs
- Rows are **typed**, according to a schema
- Cassandra does not allow data-schema incompatibily

No joins, no FKs ? Design schema accordingly

- Cassandra does not support joins (NoSQL heritage)
- Consequence: it is preferable to group / combine the data as much as possible in rows
- Schema design should take into account the most frequent queries in the workload
- Even if this may mean representing the same information in several ways (redundancy)
- Example: all movies with their directors and actors, all actors with the movies they played in, etc

Examples – defining the schema

- Creating a database (keyspace)

CREATE KEYSPACE IF NOT EXISTS Movies WITH REPLICATION ={'class': 'SimpleStrategy','replication_factor': 3};

- Creating a table (no nesting)

CREATE TABLE Artists (id text, last_name text, first_name text, birth_date int, primary key(id));

Insertions

- SQL style:

INSERT INTO artists (id, last_name, first_name, birth_date) VALUES ('artist1', 'Coppola', 'Sofia', 1971);

INSERT INTO artists (id, last_name, first_name) VALUES ('artist3', 'Marceau', 'Sophie');

- From JSON data:

INSERT INTO artists JSON '{"id":"1" ,"last_name": "Coppola","first_name": "Sofia","birth_date": "1971"}';

Nested types

- Creating a nested type:

CREATE TYPE artist (id text, last_name text, first_name text, birth_date int);

CREATE TABLE movies (id text, title text, year int, genre text, country text, director frozen<artist>, primary key(id));

Inserting data

```
INSERT INTO movies JSON '{
"id": "movie:2",
"title": "Vertigo",
"year": "1958",
"genre": "drama",
"country": "USA",
"director": {"id": "artist:3",
                "last_name": "Hitchcock",
                "first_name": "Alfred",
                "birth_date": "1899"}}';
```

Full movie table

```
CREATE TABLE movies(
                    id text,
                    title text,
                    year int,
                    genre text,
                    country text,
                    director frozen<artist>,
                    actors set< frozen<artist>>,
                    primary key(id) );
```

Some example queries (CQL)

- select * from artists;
- select * from artists limit 20;
- select JSON * from artists;
- select * from artists where id='artist:31’;
- select * from artists where last_name='Cruise’ ; NO
- select title from movies;
- select title, director.last_name from movies;
- select title, actors.last_name from movies; NO
- select cast(year as text) as yearText from movies ;
- select count(*) from movies ;

Why CQL is not SQL (1)

- Why a condition on a non-key attribute is rejected? It's all about the organization of the data: Cassandra organizes a table according to a structure which allows very quickly to find a document by its key
- This structure exists only for the key
- Any search on another attribute has no alternative but to sequentially traverse the entire table by testing the condition

Why CQL is not SQL (2)

- select * from artists where id > 'hhh’; NO
- select * from artists order by id; NO
- select * from movies order by title; NO
- select * from artists where last_name='Cruise' allow filtering; YES
- select * from artists where last_name='Cruise' and first_name='Tom' allow filtering; YES

Secondary indexes

- create index on movies(year);
- select * from movies where year = 1992; YES

Why CQL is not SQL (3)

- In general, the cost of evaluating a query is proportional to the **size of the database**
- Cassandra tries to limit the queries to those whose cost is proportional to the **size of the result**
- These restrictions must be interpreted in a Big Data context

Consistency levels in Cassandra (read)

- ONE (TWO , THREE) → the coordinator receives an answer from the first replica and sends it to the client
  - High availability, low latency, at the expense of consistency (copy may not by synchronized with others)
- QUORUM → the coordinator receives answers from at least $\lfloor replication/2 \rfloor$+1 replicas
  - Best tradeoff
- ALL → the coordinator receives answers from all replicas
  - If one does not answer, the query fails (consistency at the expense of availability)

Consistency levels in Cassandra (write)

How many ACKs the coordinator must receive from the replicas before confirming the write? Tradeoff availability/latency – consistency

- ANY : if all nodes are inactive, Hinted Handoff is applied: the write is done in a RAM buffer, waiting for a new try later
  - Minimal consistency, as data not readable, coordinator may fail
- ONE (TWO , THREE) : the client's write is confirmed if the coordinator receives an ACK from at least one replica (two, three)
- QUORUM : the client's write is confirmed if the coordinator receives ACKs from at least $\lfloor replication/2 \rfloor$+1 replicas
- ALL : the client's write is confirmed if the coordinator receives ACKs from all replicas

Tunable consistency

Write(W)			Read(R)	
Level	Description		Level	Description
ZERO	Cross fingers		N/A	
ANY	1st Response (Including HH)		N/A	
One	1st Response		One	1st Response
QUORUM	N/2 + 1 Replicas		QUORUM	N/2 + 1 Replicas
ALL	All Replicas		ALL	All Replicas

Consistency levels in summary

- R is the number of replicas consulted during Read
- W is the number of replicas consulted during Write
- RF is the replication factor

Cassandra can be made fully consistent under the following condition **R +W > RF**

- QUORUM on reads and writes meets this requirement
- If lower latency is required, one may choose R +W <= RF

Examples:

- W=ALL and R=1 : one read enough, but all writes are in sync, so we are sure to have the latest version
- W=1 and R=ALL: same

Conclusion

- Cassandra is a very fast and scalable database ... in the right context !
  - May be able to give you answers fast – if you can manage the difficulty in querying data – but no guarantee you get the correct answer in real-time
- Excels at write performance: ideal for workloads of append-only operations

Cassandra not good for

- Low latency queries under strong consistency
- Relational data (normalized) and joins
- Transactional operations (Rollback, Commit)
- Atomicity guarantees across multiple keys
- Distributed transactions

Future extensions ?

Reading further on Cassandra

- **Cassandra: The Definitive Guide, 2nd Edition,** O-Reilly media